Trajectory State Model-based Reinforcement Learning for Truck-trailer Reverse Driving

Hao Yan*
Electrical and Computer Engineering Department
Oakland University
Rochster, USA
haoyan@oakland.edu
*Corresponding author

Mohamed A. Zohdy
Electrical and Computer Engineering Department
Oakland University
Rochster, USA
zohdyma@oakland.edu

Edrees Alhawsawi
Electrical and Computer Engineering Department
Oakland University
Rochster, USA
ealhawsawi@oakland.edu

Amr Mahmoud
Electrical and Computer Engineering Department
Oakland University
Rochster, USA
amahmoud@oakland.edu

2024 8th International Conference on Robotics, Control and Automation (ICRCA) | 979-8-3503-4472-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICRCA60878.2024.10649133

***Abstract*—Maneuver the truck-trailer wheeled robot (TTWR) system for reverse trajectory following is a challenging task even for experienced drivers, which requires advanced control strategies to ensure system stability, steering precision, and driving safety. In this paper, we present the integration of a small size network model with trajectory planning and reference state prediction to achieve remarkable sample efficiency compared with pure model-free reinforcement approaches. By employing the trajectory state prediction model as the foundation model to interact with the Proximal Policy Optimization (PPO) framework, we enable reversible access to the Markov Decision Process (MDP) dynamics and help the PPO to better estimate vehicle dynamics used for controller. A numerical simulation is conducted to demonstrate the trajectory following accuracy of the proposed methodology compared with popular industry control methods such as linear–quadratic regulator (LQR). Our results indicate that the proposed hybrid trajectory model based approach not only reduces the need for extensive data collection but also achieves similar control accuracy, suggesting a promising direction for future research in autonomous vehicle control, emphasizing the need for efficient, adaptable, and robust learning algorithms.**

***Index Terms*—model-based reinforcement learning, trajectory following, trailer control, advanced driver-assistance system**

I. Introduction

In the rapidly evolving research filed of vehicular technology, Advanced Driver Assistance Systems (ADAS) have emerged as a revolutionary product, aiming to redefine daily commute and logistics through enhanced safety, active driving support, and improved fuel efficiency. In logistics industry, which is one of the key end-use industries for ADAS products, the global market size of TTWR to be valued at USD 29.36 billion by 2028 and is expected to grow at a compound annual growth rate of 5.8% during the forecast period [2]. However, the intricate challenges posed by complex road geometries, multi-agent interactions, and the need for precise integration between multiple sensors and perception, planning and control algorithms. Particularly in the case of a TTWR system, the intricate dynamics characterized by nonlinearity, nonholonomic constraints and under-actuation, compounded by the extensive blind zone at the trailer's rear, result the development of a reverse driving control algorithm for TTWR exceptionally to be more challenging [3].

However, the intricate challenges posed by complex road geometries, multi-agent interactions, and the need for precise integration between multiple sensors and perception, planning and control algorithms. Especially for logistics industry, navigating the complexities of semi-trailer reverse trajectory following presents a formidable challenge due to the nonlinear, nonholonomic, unstable, and under-actuated system behaviors of TTWR system. Since last century, trajectory planning and motion control of TTWR as a long-lasting topic has been broadly studied and lead to the creation of a variety of control strategies. Nguyen et al. showcased neural networks autonomously controlling nonlinear systems, using the truck backer-upper as an example. The system employed a two-layer neural network emulator for accurate kinematic modeling and a nonlinear controller network for managing the truck-trailer dynamics [1]. A Linear Quadratic controller for a 2-trailer system was built to ensure stable backward trajectory following, accommodating disturbances and off-axle hitching, validated by linear differential inclusion theory and optimized via a unified Lyapunov function [4]. Evestedt et al. introduces a framework combining an LQ-controller and pure pursuit trajectory tracker for automated planning and execution of reverse maneuvers with a dolly steered trailer, validated through Monte Carlo simulations and real-world tests on a scaled platform [5]. Liu et al. proposes an active articulated vehicle

Authorized licensed use limited to: University of Maryland College Park. Downloaded on October 13,2026 at 05:26:52 UTC from IEEE Xplore. Restrictions apply.

structure with a steering motor and mecanum wheels, employing a two-level coordinated control method combining Model Predictive Control (MPC) and Sliding Mode Control (SMC), enhancing maneuverability and robustness, as demonstrated by simulations [6].

Advancements in computer science, information science, and sensor technology have significantly simplified and reduced the cost of data acquisition, communication, computation, and the integration of machine learning algorithms into ADAS has become a prominent development within the automotive sector. Deep reinforcement learning, in particular, has emerged as a promising approach for addressing the complex decision-making and control challenges in continuous control development, offering adaptability and enhanced performance through iterative learning [7]. Hassani et al. leverages smart sensors and the model-free data-driven unfalsified control methodology to directly utilize data in Dynamic Positioning (DP) systems, offering a novel approach for marine system control without traditional modeling [8]. Liao et al. introduces a redefined output model-free adaptive control (RO-MFAC) method for autonomous surface vehicle heading control, addressing the unique dynamics of such systems, ensuring stability, and demonstrating effectiveness through simulations and field experiments [9].

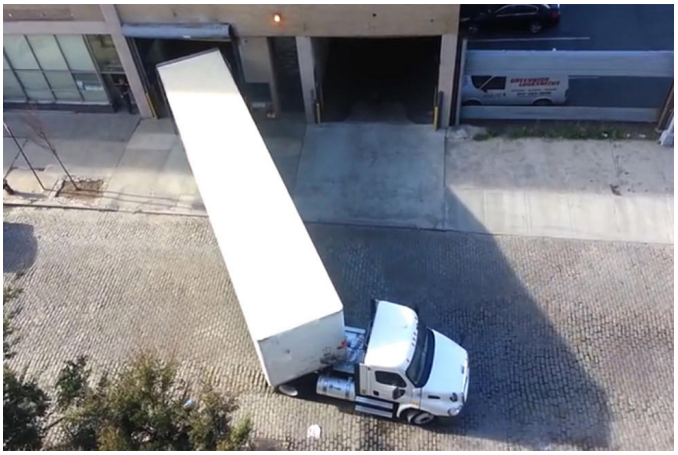


Fig. 1. A truck reverse driving towards the loading dock

While model-free deep reinforcement learning has achieved remarkable successes across various domains, its application remains limited by substantial sample inefficiency and a lack of interpretability. Model-based reinforcement learning algorithms are generally regarded as being more efficient, the model-based policy search methods tackle the challenge of sample inefficiency by utilizing observed trajectories to construct a predictive model of the robot's dynamics and interactions with its surroundings [11]. However, previous works have relied on straightforward function approximators [14] or Bayesian models [15] to maintain sample efficiency and prevent overfitting. The application of these methods to diverse and intricate high-dimensional tasks is challenging. Efforts to address these issues have included the use of large-scale neural networks to capture the intricate dynamics typical of deep reinforcement learning benchmarks. However, these models often require extended training periods and struggle with transferability and interpretability [12], which confines the application scope of these models to tasks with relatively static scenarios and assignments.

In our study, we illustrate that the pure PPO algorithm can significantly enhance sample efficiency within a trajectory state model as framework foundation, which enables the TTWR system to effectively execute trajectory planning and tracking tasks. This model assimilates environmental constraints and vehicle kinematics, employing traditional planning and predictive models to calculate the vehicle's current state and predict subsequent states. These predictions are then fed into the PPO network to facilitate model training. The simulation results show the integrated trajectory state model-based reinforcement learning algorithm not only expedites the learning process but also enhances sample efficiency and and transferability of the model. Moreover, it follows to the idea of model-based reinforcement learning by iteratively refining the system model through continuous interaction with the environment, thereby optimizing the decision-making process for complex driving maneuvers.

II. Related work of model-based reinforcement learning

Model-free deep reinforcement learning including Deep Q-Networks [17] [18], Policy Gradients [19], Actor-Critic methods [20], and Trust Regions [21] have been developed and gained popularity for their proficiency in mastering complex skills within extensive, high-dimensional state spaces, covering a range of applications from simulated robotic movement and vehicle driving to computer gaming and stock prediction. However, compared to model-free approaches, model-based reinforcement learning (MBRL) algorithms have its advantages in terms of model transferability because of inherent understanding of environmental and agent dynamics. This comprehension facilitates generalization across similar tasks, allowing MBRL to adapt to new challenges with better performance [22]. The data efficiency of MBRL is particularly beneficial when transitioning to new tasks, as it requires fewer interactions to learn and adjust to new dynamics [23]. Moreover, MBRL's ability to simulate environments aids in sim-to-real transfers, enabling policies trained in simulated models to be fine-tuned with real-world data. The explicit models used in MBRL also offer greater interpretability, allowing for easier modifications for new tasks and more straightforward troubleshooting. While the transferability of MBRL is contingent on the model's ability to generalize, certain model-free methods can also exhibit transferability, especially when enhanced by meta-learning or domain randomization. The superiority of MBRL in transferability ultimately depends on the specific application and the methodologies employed in training and implementation. TreeQN integrates the model-free reinforcement learning with online planning, outperforming existing methods in complex tasks, the proposed network

Authorized licensed use limited to: University of Maryland College Park. Downloaded on October 13,2026 at 05:26:52 UTC from IEEE Xplore. Restrictions apply.

looks like a standard value network, but it is constructed and behaves for planning purpose. By constructing and utilizing a recursive tree structure for decision-making, these methods optimize learned models for planning, demonstrating superior performance in both simulated and gaming environments, with deeper trees yielding better results [25]. MuZero, an innovative algorithm, unfolds a model to forecast a sequence of values contingent on actions taken. Subsequently, the graph is refined to enhance its capability to anticipate a model-free value assessment at every stage. The algorithm merges tree-based search with a learned model to excel in complex domains, achieving superhuman results in 57 Atari games and matching AlphaZero in Go, chess, and shogi, all without prior dynamics knowledge [26].

Additionally, drawing from the principles of transfer learning, where knowledge from a source task accelerates learning in a new task, model-based reinforcement learning employs various methods to enhance the model transferability, effectively leveraging past insights to improve performance on novel challenges. Model-based approach gets transferability by utilizing consistent dynamics across different reward structures [24], or by adapting to tasks with subtly varied dynamics which is achieved by segregating the state into an directly transferable called 'agent space' and a characterizes the new task called'problem space' [28]. Plan2Explore, a self-supervised reinforcement learning agent, innovatively explores by planning for future novelty and adapts swiftly to new tasks, demonstrating near-oracle performance on complex control tasks from visual inputs, even without task-specific training or interaction [27].

Moreover, the significance of Explainable AI (XAI) is increasingly recognized in the AI sphere, prompted by the imperative to comprehend AI systems' decision-making, particularly in vital sectors involving human-machine interaction. In 2020, research [29] introduced an autonomous vehicle controller with 19 neurons, showcasing its interpretability against more opaque, larger models. Additionally, advancements in MBRL [30] have led to agents capable of explaining their actions via predicted state transitions, offering user-friendly descriptions of complex states and actions. This method contrasts user-specific queries with the agent's policy, aiding users to grasp the overarching strategy of the AI, a feature favored according to initial surveys.

III. TTWR Trajectory Planning

A. Frenet-Serret frame

One of the main problem in robotics is to determine the minimum time trajectory from an initial pose to a target pose, taking into account various constraints on the vehicle such as limited curvature, velocity bounds, and acceleration limits. The trajectory curve can be generated by moving the orthonormal frame of vehicle coordinate system (VCS), and each trajectory point can be sampled as the coordinate in world coordinate system (WCS) as well as the rotation angle of the VCS. As defined in [35], these frames are called Frenet-Serret frames. In Figure 2, consider a point Q on the curve $r(t)$, at

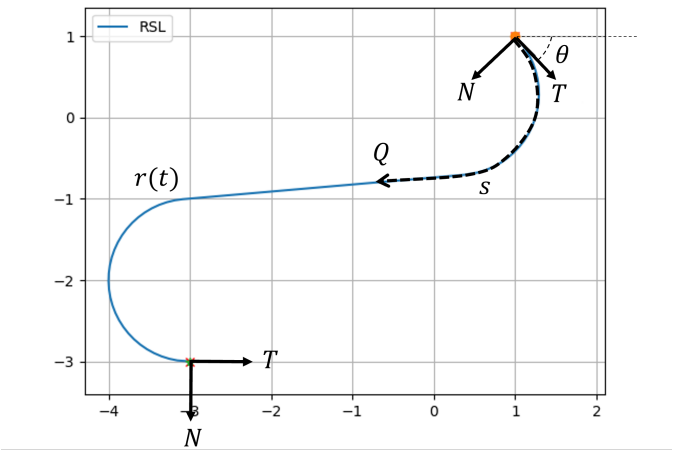


Fig. 2. In Frenet-Serret frame, the Dubins path starting from point (1, 1) with heading angle $\frac{-\pi}{4}$ and ending at (-3, -3) with heading angle 0

point Q, the unit tangent vector (x axis of the VCS) is denoted as $\mathbf{T}$ and the unit normal vector (y axis of the VCS) as $\mathbf{N}$. The tangent $\mathbf{T}$ forms an angle γ with the x-axis in WCS, which is the rotation angle of the VCS, and the arc length of the trajectory at point Q is denoted by s. The progression of the frame along the trajectory, as well as the equations that describe the frame at point Q, are governed by the following relationships:

$$\begin{aligned} T' &= \kappa(s)N = \frac{dT}{ds} \\ N' &= -\kappa(s)T = \frac{dN}{ds} \\ T &= \frac{dx}{ds} = \cos(\theta) \\ N &= \frac{dy}{ds} = \sin(\theta) \\ \kappa &= \frac{d\theta}{ds} \end{aligned} \tag{1}$$

These equations encapsulate the dynamic evolution of the Frenet–Serret frame as the TTWR system navigates its trajectory, providing a mathematical foundation for understanding and predicting the system's movement through space.

B. Dubins Path

To generate the trajectory state for RL agent to learn, the Dubin path is constructed and sampled before the agent learning. Previous research [36] shows the shortest trajectory for a curvature-constrained vehicle between two points, without any intervening obstacles, is limited to a starting arc followed by a straight line and ending with another arc . Each path's unique characteristics are determined by three parameters: the angles of the arcs and the length of the straight line. Further improvements to the Dubins path is be achieved by changing the curvature limitation and integrating time efficiency and risk evaluation as cost value into the trajectory planning algorithm,

Authorized licensed use limited to: University of Maryland College Park. Downloaded on October 13,2026 at 05:26:52 UTC from IEEE Xplore. Restrictions apply.

which is especially important in obstacle-dense environments [37]. Consider the following inputs and constrains:

1) Initial pose of the target: $P_s\,(x_s, y_s, \theta_s)$
2) Final pose of the target: $P_f\,(x_f, y_f, \theta_f)$
3) Initial turning radius: $R_s = \frac{1}{\kappa_s}$
4) Final turning radius: $R_f = \frac{1}{\kappa_f}$

where the initial turning radius and final turning radius is equal for vehicle trajectory planning, which is determined by vehicle steering angle:

$$R = \frac{L}{tan(\delta)} \tag{2}$$

To compute the Dubins path, we need to start from the two turning circle center $O_s\,(x_{cs}, y_{cs})$ and $O_f\,(x_{cf}, y_{cf})$, which can be computed as:

$$(x_{cs}, y_{cs}) = \tag{3}$$
$$(x_s \pm \rho_s \cos\left(\theta_s \pm \frac{\pi}{2}\right), y_s \pm \rho_s \sin\left(\theta_s \pm \frac{\pi}{2}\right))$$

$$(x_{cf}, y_{cf}) = \tag{4}$$
$$(x_f \pm \rho_f \cos\left(\theta_f \pm \frac{\pi}{2}\right), y_f \pm \rho_f \sin\left(\theta_f \pm \frac{\pi}{2}\right))$$

The reference circle of radius $|\rho_f - \rho_s|$ is constructed at O_f for $\rho_s \leq \rho_f$, and the two primary circle center is connected with center reference line:

$$|c| = \sqrt{(x_{cs} - x_{cf})^2 + (y_{cs} - y_{cf})^2} \tag{5}$$

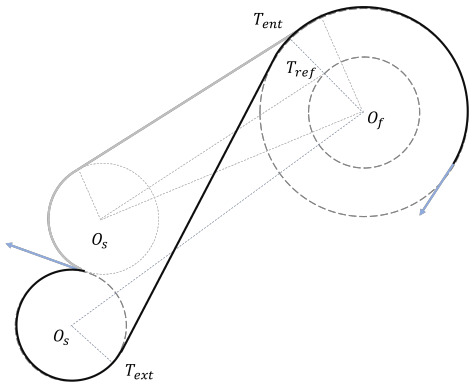


Fig. 3. Dubins path construction following LSR pattern, for the same start and finish point, the Dubins path following RSR pattern is in grey color for comparison

The resulting composite path consists of the starting arc P_sT_{ext}, the external tangent line $T_{ext}T_{ent}$, and the ending arc $T_{ent}P_f$. To determine the shortest Dubins path between the same initial and final points, multiple patterns are considered in Figure 3, and the one with the least total length is selected.

IV. TTWR STATE SPACE MODEL

A. TTWR kinematics model

Extensive prior works [31] [32] have explored various models for analyzing vehicle states, ranging from kinematic to dynamic models. The selection of a suitable model is typically determined by the required precision and the driving conditions. For tasks require high-speed driving, dynamic elements are crucial for motion planning and control. In contrast, kinematic constraints become more prominent in low-speed scenarios, affecting the motion of the TTWR system more significantly. Given that TTWR reverse parking in this paper is a low-speed maneuver, our study constructs the system model on kinematic principles. We adopt an Ackermann steering mechanism for the truck and a single-axle for the trailer, forming our TTWR system, which is also the most common user case in North America market. These two components are connected via a hitch ball, serving as a pivotal joint allowing for their relative rotational movements. Mechanically, the truck and trailer are considered distinct rigid bodies, each exhibiting axial symmetry, with their left and right sides being mirror images. When operating on level, high-traction surfaces at low velocities, factors such as tire side-slip and delays in control response from mechanical or electrical systems are minimal and can be disregarded [33]. The key parameters defining the TTWR system are summarized in Table I.

TABLE I
THE TTWR SYSTEM STATES

Parameters	Descriptions
x_1	Truck lateral position in WCS
y_1	Truck longitudinal position in WCS
θ_1	Truck yaw angle
x_2	Trailer lateral position in WCS
y_2	Trailer longitudinal position in WCS
θ_2	Trailer yaw angle
ϕ	The truck-trailer angle
v	Truck longitudinal velocity
δ	Truck steering angle
L_1	Truck wheelbase length
L_2	Truck rear overhang, defined from the rear axle to the hitch point
L_3	Trailer overhang, defined from the the hitch to trailer axle

In the world coordinate system, the truck's state model is based on the front wheel Ackerman steering principle, which governs the steering angle of the front wheels relative to the truck's heading angle. Due to computational constraints, vehicle dynamics are often greatly simplified during simulation [34]. Under the low-speed conditions and the assumption of an ideal flat surface for this study, the full track vehicle model in [16] can be reduced to the kinematic bicycle model in Figure 4. The kinematic equations for the truck vehicle are as follows:

$$\dot{x}_1 = v\cos\theta_1 \tag{6}$$

$$\dot{y}_1 = v\sin\theta_1 \tag{7}$$

$$\dot{\theta}_1 = \frac{v}{L_1}\tan\delta \tag{8}$$

The kinematic model for the trailer is derived from the truck's state space, as the trailer's movement is contingent on the truck's actions. The trailer's longitudinal velocity is calculated by projecting the truck's normal and longitudinal velocities onto the trailer's longitudinal axis at the hitch point.

Authorized licensed use limited to: University of Maryland College Park. Downloaded on October 13,2026 at 05:26:52 UTC from IEEE Xplore. Restrictions apply.

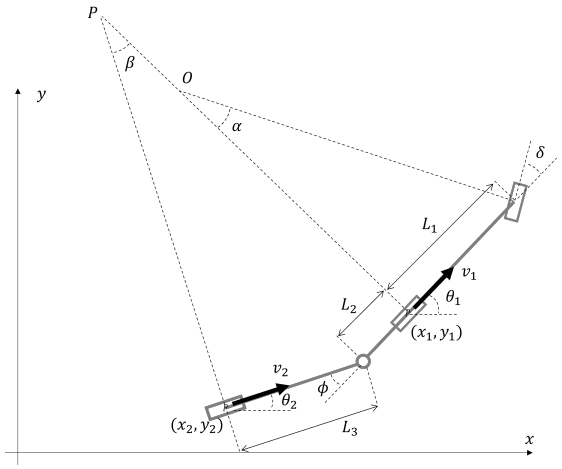


Fig. 4. Simplified bicycle kinematic model of TTWR

In line with the bicycle model, the trailer's states $[x_2, y_2, \theta_2]^T$ are given by:

$$\dot{x}_2 = v\cos\phi(1 - \frac{L_2}{L_1}\tan\phi\tan\delta)\cos\theta_2 \tag{9}$$

$$\dot{y}_2 = v\cos\phi(1 - \frac{L_2}{L_1}\tan\phi\tan\delta)\sin\theta_2 \tag{10}$$

$$\dot{\theta}_2 = -v(\frac{\sin\phi}{L_3} + \frac{L_2}{L_1 L_3}\cos\phi\tan\delta) \tag{11}$$

The truck-trailer angle ϕ, crucial for predicting trailer state and assessing jack-knifing risk, is determined by the difference between Eqn.11 and Eqn.8:

$$\dot{\phi} = -\frac{v}{L_3}\sin\phi - \frac{v}{L_1}(1 + \frac{L_2\cos\phi}{L_3})\tan\delta \tag{12}$$

In reverse maneuvers, the truck-trailer system's control is confined to a single degree of freedom, which is the steering input of the truck. The control system is primarily designed to minimize the trailer's tracking error. Therefore, the control reference variables are the trailer's rear axle center position (x_2, y_2), and heading $\theta 2$, in relation to the parking slot's reference position (x_{final}, y_{final}), and heading angle θ_{final}.

B. Reference trajectory following model

Once the TTWR kinematics model generates the current vehicle states, the current state variables are applied to the predefined Dubins reference path in trajectory planning module to compute the deviation states. The lateral dynamics of the TTWR can be further simplified and characterized in Equation 13, providing an efficient description of the vehicle's kinematic behavior:

$$\begin{bmatrix}\Delta\dot{\theta}_1\\ \Delta\dot{\theta}_2\\ \Delta\dot{y}\end{bmatrix} = \begin{bmatrix}0 & 0 & 0\\ \frac{v}{L_3} & -\frac{v}{L_3} & 0\\ 0 & v & 0\end{bmatrix}\begin{bmatrix}\Delta\theta_1\\ \Delta\theta_2\\ \Delta y\end{bmatrix} + \begin{bmatrix}\frac{v}{L_1}\\ -\frac{L_2}{L_1 L_3}v\\ 0\end{bmatrix}\delta \tag{13}$$

As shown in Figure 5, the reference states error consist of $\Delta\theta_1$, $\Delta\theta_2$, and Δy, which represents the heading error between Truck heading angle and the reference point heading angle, the heading error between trailer heading angle and reference point heading angle, and the lateral distance in trailer coordinate between trailer position and reference point.

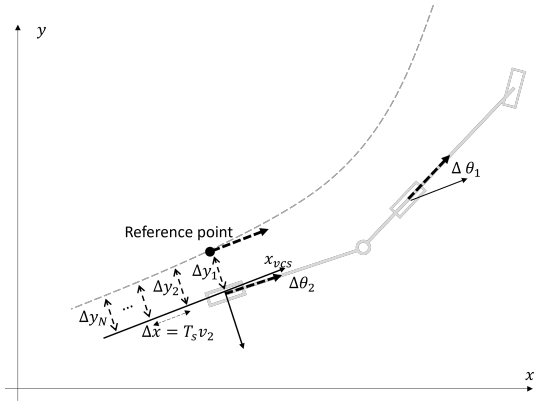


Fig. 5. Vehicle kinematics model with reference

In real control problem, preview distance is also applied to the system to improve driving consistency. In Figure 5, the first N preview reference points and the deviation value are plotted within the trailer VCS, and the reference error value for each preview reference sample is computed and denoted by $(\theta_{1_k}, \theta_{2_k}, y_k)$, Δx is the sampling interval and T_s is the simulation step time. Moreover, the Equation 13 need to be discretized in simulation:

$$\begin{aligned}\mathbf{A}_d &= \mathrm{e}^{\mathbf{A}T}\\ \mathbf{B}_d &= \mathbf{A}^{-1}\left(\mathrm{e}^{\mathbf{A}T} - \mathbf{I}\right)\mathbf{B}\end{aligned} \tag{14}$$

Therefor, the N preview sample points can be computed by:

$$\begin{bmatrix}\Delta\theta_1\\ \Delta\theta_2\\ \Delta y\end{bmatrix}_k + 1 = \mathbf{A}_d\begin{bmatrix}\Delta\theta_1\\ \Delta\theta_2\\ \Delta y\end{bmatrix}_k + \mathbf{B}_d\delta_k \tag{15}$$

where

$$A_r = \begin{bmatrix}0 & 1 & 0 & \cdots & 0\\ 0 & 0 & 1 & \cdots & 0\\ \vdots & \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & 0 & \cdots & 1\\ 0 & 0 & 0 & \cdots & 0\end{bmatrix}_{(N\times N)} \qquad B_r = \begin{bmatrix}0\\ 0\\ 0\\ \vdots\\ 1\end{bmatrix}_{(N\times 1)}$$

V. Trajectory State Model-Based Reinforcement Learning Algorithm

We now introduce our trajectory state model-based reinforcement learning algorithm. To bridge model-based algorithm's efficiency gap, we propose a synergistic approach that integrates PPO with our trajectory following state model. This integration leverages the computational efficiency of classical trajectory planning and state prediction models which forecasts upcoming system states. Our method integrates the strengths of state space model with PPO by training a policy that emulates the behavior of a LQR, resulting in a powerful and efficient hybrid learning mechanism.

Authorized licensed use limited to: University of Maryland College Park. Downloaded on October 13,2026 at 05:26:52 UTC from IEEE Xplore. Restrictions apply.

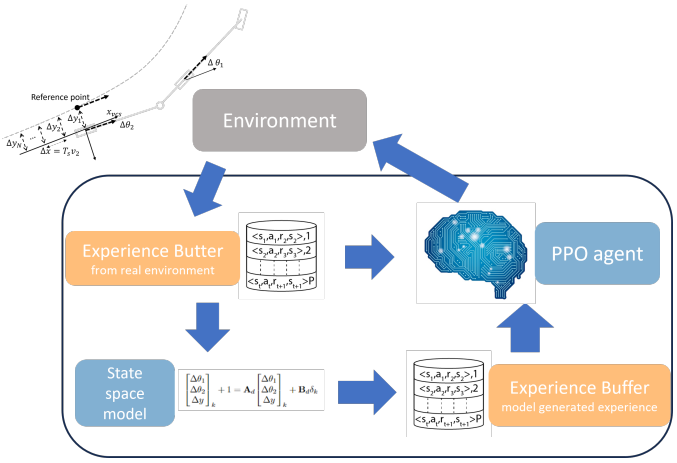


Fig. 6. Trajectory state model-based reinforcement learning agent

A. Foundation model for trajectory state prediction

We build the trajectory state model as the foundation model for neural network, which can be parameterized as $f_\theta(s_t, a_t)$, where the internal states s_t are the tracking error between trailer reference point and trajectory reference points, which is denoted by $(\theta_{1_k}, \theta_{2_k}, y_k)$, and the action input of the foundation model is truck steering angle Δx. The trajectory state model shall take the input of current state s_t and action a_t, and based on real-time environment states, the trajectory from previous cycle will be optimized based on current environment scene, and the predicted next states $\hat{s}_{t+1}$ are generated which are reversible access to the MDP dynamics.

But the model performance will be effected if the sampling time Δt becomes samller as the state transition is trivial. To address this issue, the trajectory state model is modified as: $\hat{s}_{t+1} = s_t + f\theta(s_t, a_t)$. It is important to note that a larger Δt can enhance the richness of each data sample, aiding both the learning of dynamics and subsequent planning; however, a larger Δt may also amplify the discretization errors, which potentially make the PPO agent learning process difficult to converge. The complete reward function of the trajectory state following model includes all 4 parts, as expressed in Eqn. 16:

$$R = R_{\text{tracking error}} + R_{\text{collision}} + R_{\text{parking}} + R_{\text{traveling distance}} \quad (16)$$

The tracking error reward is formulated using a Gaussian function, designed to yield the maximum reward when the tracking error is zero and to decrease as the error increases. The reward function is given by:

$$R_{\text{tracking error}} = e^{-(\Delta X^T W \Delta X)} \quad (17)$$

where ΔX is tracking errors vector specified in Equation 13, and W is a positive-definite weight matrix that scales the contribution of each tracking error component. This formulation ensures that the agent is encouraged to minimize the tracking error, aligning the agent's trajectory closely with the desired trajectory states.

B. Recurrent proximal policy optimization motion planning

After the setup of the trajectory state prediction model $f_\theta(s_t, a_t)$, which serves as a foundation for a model-free reinforcement learning algorithm. We employ Proximal Policy Optimization as the adaptive controller which tracks the TTWR trajectory following error. Here, policy gradient methods like PPO are well-suited for model-free refinement as it alternates between sampling data through interaction with the environment and optimizing a surrogate objective function using stochastic gradient ascent. PPO aims to take the most possible step to improve policy performance, without stepping so far that it accidentally causes performance to collapse. This is achieved by keeping the new policy close to the old policy:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t \right) \right] \quad (18)$$

where $r_t(\theta)$ represents the probability ratio $\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$, $\hat{A}_t$ is the advantage function estimator at timestep t, and ϵ is a hyperparameter that defines the clipping range, typically set around 0.2.

Also, the PPO algorithm is frequently selected for the benchmark tasks in robotics, offering a straightforward basis for contrasting pure model-free learning against model-based pre-initialization strategy. Utilizing PPO with trajectory planner $f_\theta(s_t, a_t)$ as the foundation model for PPO, instead of starting with a random pure model-free policy, it blends the model-based and model-free techniques, although straightforward, it proves the effective in simulation results. We also compare this approach with Recurrent PPO (RPPO) and LQR methods, showcasing the relative strengths of each method in terms of learning efficiency and controller robustness.

VI. Experimental Results

Our initial investigation focused on assessing different configurations for trajectory state model reinforcement learning, using neural networks to compare the benchmark LQR methodology. For this purpose, we utilized Gymnasium, a versatile Python library providing a wide range of standardized environments tailored for the development and comparison of reinforcement learning algorithms. Gymnasium's user-friendly interface streamlines simulation, design, testing, and benchmarking of RL models across numerous tasks and scenarios. Specifically, we developed a customized truck trailer simulation within Gymnasium to investigate reverse driving techniques with varied trailer setups. An example of these simulated trajectory states is illustrated in Figure 7. After multiple vehicle configuration with various Dubins path combination was evaluated, we used the best outcome of that evaluation for the remainder of the evaluations. The code implementation can be found: https://github.com/hynpu/dubins_trajectory_mbrl_ttwr.

In our study, we evaluate the effectiveness of the proposed learning strategy using the benchmark platform across various vehicle setups for trajectory tracking tasks. The initialization of the trajectory planning and motion prediction models were

Authorized licensed use limited to: University of Maryland College Park. Downloaded on October 13,2026 at 05:26:52 UTC from IEEE Xplore. Restrictions apply.

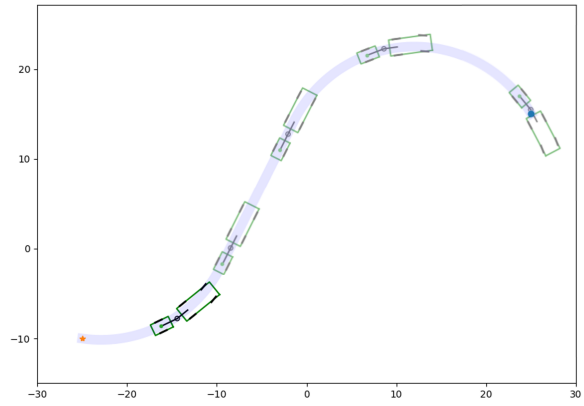


Fig. 7. Benchmark systems constructed in Gymnasium, demonstrating agents equipped with trajectory planning and motion prediction models. The figure illustrates the integration of our model-based approach with these agents to showcase performance efficacy.

based on the kinematic state space model of the vehicles which requires a single calibration for each agent. Subsequently, these models networks can be transferred and re-trained during operation to manage distinct tasks for changing vehicle configurations. The adaptability of our method was evident as it successfully handled new challenges during testing, such as managing trailers with increased overhang length which introduced more intricate dynamics.

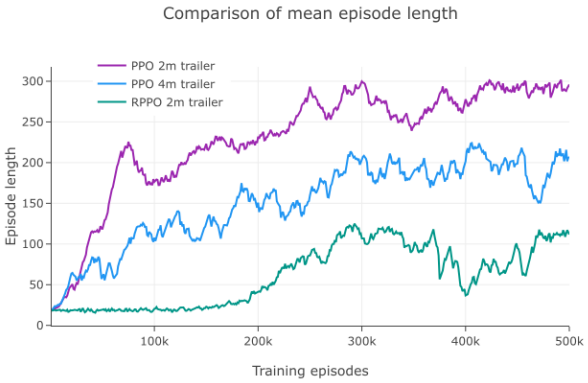


Fig. 8. Mean episode length comparison between RPPO training on 2 meters trailer, PPO training on both 2 meters and 4 meters trailer

Figure 8 compares the mean episode lengths for PPO-trained agents with 2-meter and 4-meter trailers as well as an RPPO-trained agent trained with 2-meter trailer. The PPO agent trained with a shorter trailer shows better performance, achieving efficient parking after 250,000 training steps. In contrast, the longer trailer brings more complexity for reverse driving maneuvering, which leads to a slower training, although it achieves similar parking success rate and rewards in 9. However, the success rate of RPPO algorithm is under 50% and the training takes more steps, possibly due to vanishing gradients and the intricate nature of the task's delayed rewards.

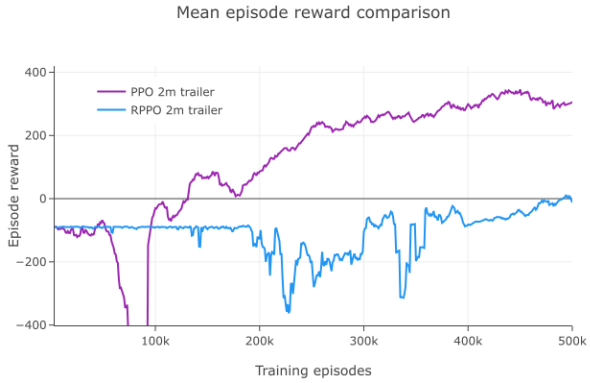


Fig. 9. Mean episode training reward comparison between PPO training on both 2 meters and 4 meters trailer

Figure 9 shows that the PPO with a 2-meters trailer is able to achieving stable rewards by 250,000 steps. The training of 4m trailer's catches up by 300k steps, reflecting the added complexity in controlling longer trailers and the consequent need for extended training to refine the parking maneuvers.



Fig. 10. Success rate of model free learning using PPO for controlling 2 meters trailer in long distance perpendicular parking scenario

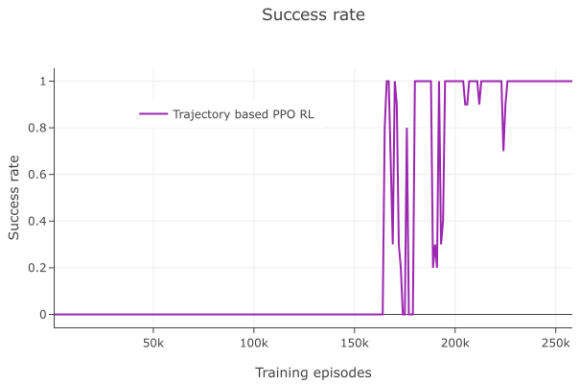


Fig. 11. Success rate of the proposed trajectory state based learning for controlling 2 meters trailer in long distance perpendicular parking scenario

To evaluate the relative efficiency of model-free and the proposed trajectory-based methods, a long-distance reverse

Authorized licensed use limited to: University of Maryland College Park. Downloaded on October 13,2026 at 05:26:52 UTC from IEEE Xplore. Restrictions apply.

parking task, typical of North American usage scenarios, was employed as shown in Figure 1. The model-free RL method, depicted in Figure 10, incrementally improves across 3 million episodes, yet exhibits significant fluctuation in success rates, suggesting considerable variance in episodic learning. In comparison, as demonstrated in Figure 11, the trajectory-based method converges rapidly, achieving substantial success rates within merely 250,000 episodes and maintaining consistent performance thereafter. This accelerated learning can be attributed to the method's generation of intermediate reference states, which likely provide the agent with more immediate and relevant targets to track for each step of the task.

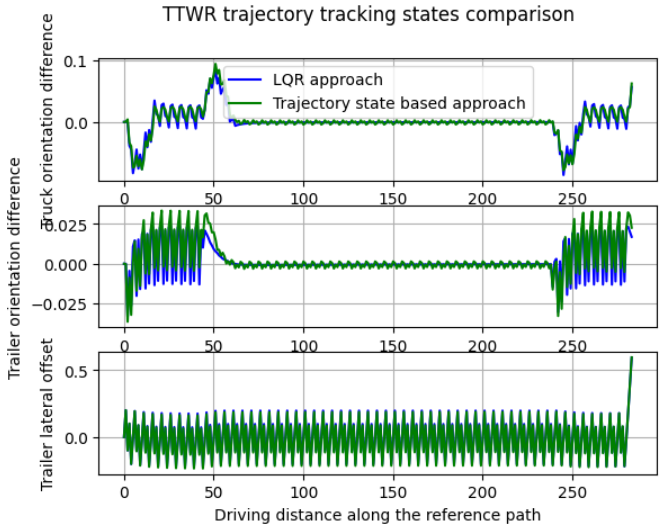


Fig. 12. The comparison of LQR controller and proposed algorithm regarding reference trajectory orientation error, trailer orientation error, and trailer lateral offset error

Figure 12 compares the LQR controller with the proposed algorithm regarding truck orientation error, trailer orientation error, and lateral offset error. For a 2-meter trailer which is popular in North America market which handles most moving tasks, the performance of both systems is closely matched, with the lateral offset consistently maintained below 0.2 meters. The result shows the proposed algorithm's high tracking accuracy and suggests its potential effectiveness in practical scenarios.

Figure 13 presents a comparative analysis of the LQR controller and the proposed algorithm, applied to a 5-meter extended trailer designed for heavy-duty tasks. Both controllers successfully follow the given trajectory; however, the proposed algorithm demonstrates slightly better performance in orientation tracking with smaller errors. This indicates enhanced adaptability of the proposed algorithm in contrast to the LQR controller, which relies on fixed feedback gains.

VII. Discussion

Our research introduces a reinforcement learning algorithm built on a trajectory state prediction model that effectively enables neural networks to learn the dynamics of TTWR reverse

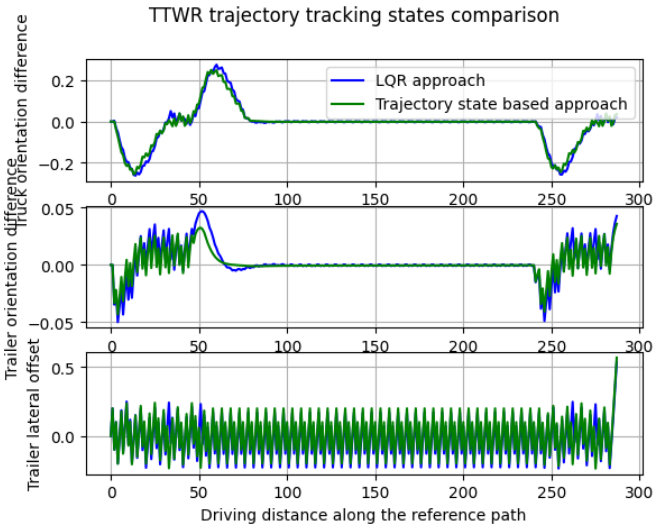


Fig. 13. The comparison of LQR controller and proposed algorithm on extended trailer (5 meters) regarding reference trajectory orientation error, trailer orientation error, and trailer lateral offset error

parking simulations with limited data samples. While previous studies have studied the neural network based learning for dynamic tasks, the simulation results show the effectiveness of the algorithm proposed for the TTWR system training. The system, once trained, exhibits competent decision-making that optimizes the total reward. It also maintains the jackknifing and obstacle collision in low occurrence rate throughout the parking maneuver, simplifies the possible forward driving adjustment maneuver and improves driver's user experience.

This research decreases the need for extensive calibration in traditional control systems while maintaining the level of precision and stability for trajectory tracking. However, there are still potential improvements that require attention and resolution. The first direction is to partially replace the current trajectory state model with neural networks to improve generalization of the model, which is inspired by the implementation of Imagination-Augmentation (IA) for autonomous driving. Also, while the simulations validate the robustness of our findings, real-world testing remains necessary for the future research.

References

[1] D. H. Nguyen and B. Widrow, "Neural networks for self-learning control systems," in IEEE Control Systems Magazine, vol. 10, no. 3, pp. 18-23, April 1990, doi: 10.1109/37.55119.

[2] Semi-trailer Market Size, Share & Trends Analysis Report by Type (Flat Bed Trailer, Dry Vans, Refrigerated Trailers, Lowboy Trailers, Tankers), by Region (North America, Europe, APAC, LATAM, MEA), and Segment Forecasts, 2021-2028

[3] H. Yan, M. A. Zohdy, A. Shaout, and A. Mahmoud, "Recurrent Proximal Policy Optimization Based Tractor-Trailer Wheeled Robot Automatic Parking Algorithm", TECS, vol. 11, no. 4, pp. 124–142, Aug. 2023.

[4] O. Ljungqvist, D. Axehill and A. Helmersson, "Path following control for a reversing general 2-trailer system," 2016 IEEE 55th Conference on Decision and Control (CDC), Las Vegas, NV, USA, 2016, pp. 2455-2461, doi: 10.1109/CDC.2016.7798630.

Authorized licensed use limited to: University of Maryland College Park. Downloaded on October 13,2026 at 05:26:52 UTC from IEEE Xplore. Restrictions apply.

[5] N. Evestedt, O. Ljungqvist and D. Axehill, "Motion planning for a reversing general 2-trailer configuration using Closed-Loop RRT," 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Daejeon, Korea (South), 2016, pp. 3690-3697, doi: 10.1109/IROS.2016.7759544.

[6] Z. Liu, M. Yue, L. Guo, and Y. Zhang, "Trajectory planning and robust tracking control for a class of active articulated tractor-trailer vehicle with on-axle structure," European Journal of Control, vol. 54, pp. 87–98, Jul. 2020, doi: 10.1016/j.ejcon.2019.12.003.

[7] F. Ye, X. Cheng, P. Wang, C.-Y. Chan, and J. Zhang, "Automated Lane Change Strategy using Proximal Policy Optimization-based Deep Reinforcement Learning," in 2020 IEEE Intelligent Vehicles Symposium (IV), Las Vegas, NV, USA: IEEE, Oct. 2020, pp. 1746–1752. doi: 10.1109/IV47402.2020.9304668.

[8] V. Hassani, A. M. Pascoal, and T. F. Onstein, "Data-driven control in Marine Systems," Annual Reviews in Control, vol. 46, pp. 343–349, 2018. doi:10.1016/j.arcontrol.2018.10.006

[9] Y. Liao, Q. Jiang, T. Du and W. Jiang, "Redefined Output Model-Free Adaptive Control Method and Unmanned Surface Vehicle Heading Control," in IEEE Journal of Oceanic Engineering, vol. 45, no. 3, pp. 714-723, July 2020, doi: 10.1109/JOE.2019.2896397.

[10] Z. Du, Q. Miao, and C. Zong, "Trajectory Planning for Automated Parking Systems Using Deep Reinforcement Learning," Int.J Automot. Technol., vol. 21, no. 4, pp. 881–887, Aug. 2020, doi: 10.1007/s12239-020-0085-9.

[11] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, "Trust Region Policy Optimization." arXiv, Apr. 20, 2017. Accessed: Nov. 02, 2023. [Online]. Available: http://arxiv.org/abs/1502.05477

[12] A. Nagabandi, G. Kahn, R. S. Fearing, and S. Levine, "Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning." arXiv, Dec. 01, 2017. Accessed: Oct. 14, 2023. [Online]. Available: http://arxiv.org/abs/1708.02596

[13] M. P. Deisenroth, "A Survey on Policy Search for Robotics," FNT in Robotics, vol. 2, no. 1–2, pp. 1–142, 2011, doi: 10.1561/2300000021.

[14] R. Lioutikov, A. Paraschos, J. Peters and G. Neumann, "Sample-based informationl-theoretic stochastic optimal control," 2014 IEEE International Conference on Robotics and Automation (ICRA), Hong Kong, China, 2014, pp. 3896-3902, doi: 10.1109/ICRA.2014.6907424.

[15] M. Deisenroth and C. E. Rasmussen, "PILCO: A model-based and data-efficient approach to policy search," in Proceedings of the 28th International Conference on machine learning (ICML-11), 2011, pp. 465–472. Accessed: Nov. 08, 2023. [Online]. Available: https://www.cs.utexas.edu/users/sniekum/classes/RLFD-F15/papers/Deisenroth11.pdf

[16] H. Yan, F. Yang and Z. Wang, "Model Predictive Control for Unmanned Tracked Vehicle Path Following," 2018 International Conference on Sensing,Diagnostics, Prognostics, and Control (SDPC), Xi'an, China, 2018, pp. 101-106, doi: 10.1109/SDPC.2018.8664956.

[17] V. Mnih et al., "Playing Atari with Deep Reinforcement Learning." arXiv, Dec. 19, 2013. Accessed: Nov. 02, 2023. [Online]. Available: http://arxiv.org/abs/1312.5602

[18] B. Peng et al., "End-to-End Autonomous Driving Through Dueling Double Deep Q-Network," Automot. Innov., vol. 4, no. 3, pp. 328–337, Aug. 2021, doi: 10.1007/s42154-021-00151-3.

[19] G. Li, S. Li, S. Li, and X. Qu, "Continuous decision-making for autonomous driving at intersections using deep deterministic policy gradient," IET Intelligent Trans Sys, vol. 16, no. 12, pp. 1669–1681, Dec. 2022, doi: 10.1049/itr2.12107.

[20] I. Grondman, L. Busoniu, G. A. Lopes, and R. Babuska, "A survey of actor-critic reinforcement learning: Standard and natural policy gradients," IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), vol. 42, no. 6, pp. 1291–1307, 2012.

[21] L. Engstrom et al., "Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO." arXiv, May 25, 2020. Accessed: Nov. 02, 2023. [Online]. Available: http://arxiv.org/abs/2005.12729

[22] A. S. Polydoros and L. Nalpantidis, "Survey of model-based reinforcement learning: Applications on robotics," Journal of Intelligent & Robotic Systems, vol. 86, no. 2, pp. 153–173, 2017.

[23] L. Kaiser et al., "Model-Based Reinforcement Learning for Atari." arXiv, Feb. 19, 2020. Accessed: Nov. 02, 2023. [Online]. Available: http://arxiv.org/abs/1903.00374

[24] C. G. Atkeson and J. C. Santamaria, "A comparison of direct and model-based reinforcement learning," Proceedings of International Conference on Robotics and Automation, Albuquerque, NM, USA, 1997, pp. 3557-3564 vol.4, doi: 10.1109/ROBOT.1997.606886.

[25] S. Whiteson, "TreeQN and ATreeC: Differentiable tree planning for deep reinforcement learning," 2018, Accessed: Nov. 02, 2023. [Online]. Available: https://www.cs.ox.ac.uk/publications/publication11671-abstract.html

[26] J. Schrittwieser et al., "Mastering Atari, Go, chess and shogi by planning with a learned model," Nature, vol. 588, no. 7839, Art. no. 7839, Dec. 2020, doi: 10.1038/s41586-020-03051-4.

[27] R. Sekar, O. Rybkin, K. Daniilidis, P. Abbeel, D. Hafner, and D. Pathak, "Planning to Explore via Self-Supervised World Models," in Proceedings of the 37th International Conference on Machine Learning, PMLR, Nov. 2020, pp. 8583–8592. Accessed: Nov. 02, 2023. [Online]. Available: https://proceedings.mlr.press/v119/sekar20a.html

[28] G. D. Konidaris and A. G. Barto, "Building Portable Options: Skill Transfer in Reinforcement Learning.," in Ijcai, 2007, pp. 895–900. Accessed: Nov. 02, 2023.

[29] M. Lechner, R. Hasani, A. Amini, T. A. Henzinger, D. Rus, and R. Grosu, "Neural circuit policies enabling auditable autonomy," Nat Mach Intell, vol. 2, no. 10, Art. no. 10, Oct. 2020, doi: 10.1038/s42256-020-00237-3.

[30] J. van der Waa, J. van Diggelen, K. van den Bosch, and M. Neerincx, "Contrastive Explanations for Reinforcement Learning in terms of Expected Consequences." arXiv, Jul. 23, 2018. doi: 10.48550/arXiv.1807.08706.

[31] F. Sorge, "Motion and force analysis of slow-speed multi-trailer systems," Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science, vol. 233, no. 8, pp. 2615–2635, Apr. 2019, doi: 10.1177/0954406218792589.

[32] X. Liu, A. K. Madhusudhanan, and D. Cebon, "Minimum Swept-Path Control for Autonomous Reversing of a Tractor Semi-Trailer," IEEE Trans. Veh. Technol., vol. 68, no. 5, pp. 4367–4376, May 2019, doi: 10.1109/TVT.2019.2895513.

[33] G. Lei and Y. Zheng, "Research on Cooperative Trajectory Planning Algorithm Based on Tractor-Trailer Wheeled Robot," IEEE Access, vol. 10, pp. 64209–64221, 2022, doi: 10.1109/ACCESS.2021.3062392.

[34] P. Polack, F. Altche, B. d'Andrea-Novel, and A. De La Fortelle, "The kinematic bicycle model: A consistent model for planning feasible trajectories for autonomous vehicles?," in 2017 IEEE Intelligent Vehicles Symposium (IV), Los Angeles, CA, USA: IEEE, Jun. 2017, pp. 812–818. doi: 10.1109/IVS.2017.7995816.

[35] M. Shanmugavel, A. Tsourdos, B. White, and R. Żbikowski, "Co-operative path planning of multiple UAVs using Dubins paths with clothoid arcs," Control Engineering Practice, vol. 18, no. 9, pp. 1084–1092, Sep. 2010, doi: 10.1016/j.conengprac.2009.02.010.

[36] L. E. Dubins, "On Curves of Minimal Length with a Constraint on Average Curvature, and with Prescribed Initial and Terminal Positions and Tangents," American Journal of Mathematics, vol. 79, no. 3, pp. 497–516, 1957, doi: 10.2307/2372560.

[37] J. Song, S. Gupta and T. A. Wettergren, "T*: Time-Optimal Risk-Aware Motion Planning for Curvature-Constrained Vehicles," in IEEE Robotics and Automation Letters, vol. 4, no. 1, pp. 33-40, Jan. 2019, doi: 10.1109/LRA.2018.2876888.

Authorized licensed use limited to: University of Maryland College Park. Downloaded on October 13,2026 at 05:26:52 UTC from IEEE Xplore. Restrictions apply.